

A2 · Stop Typing Your Password

CS425 Computer Networks · instructor led · follow along on your own laptop

TURN THIS IN ON PAPER

before you leave the room

graded pass/fail

Names _____ Date _____

Why you are doing this

Last week you typed your Onyx password a dozen times. Today you type it twice more and then never again. You also put one command, **probe**, on the box permanently; it is a quick connection test. Follow the projector. I wait at every step, so nobody is racing anybody.

STEPS 0 TO 4 · GET THERE, KEYS, CONFIG, AND WHAT THE SECOND LOGIN SAVES

```
ssh <username>@onyx.boisestate.edu          # step 0: password login must work before anything else
ssh-keygen -t ed25519 -C "you@boisestate.edu" # step 1, on your laptop
ssh-copy-id <username>@onyx.boisestate.edu    # step 2, the last password you type
ssh onyx uname -n                             # step 3, after ~/.ssh/config is in place
ssh -O exit onyx 2>/dev/null; time ssh onyx true; time ssh onyx true # step 4, cold then warm
ping -c 3 onyx.boisestate.edu                # step 4, the round trip (Git Bash: ping -n 3)
```

MEMBER	LAPTOP OS	STEP 0 PASSWORD	STEP 2 KEY	COLD SSH	WARM SSH	ROUND TRIP (PING AVG)

Step 0 failed? **command not found**: use Git Bash. **Could not resolve**: spelling or Wi-Fi. **timed out**: try a hotspot. **Permission denied**: username, not email. Still stuck: **ssh -v**, read the last lines.

Q1. The cold login cost some number of round trips. Given your cold time and your ping, roughly how many, and what was SSH doing in them? Whose speedup was largest, and why?

STEPS 5 TO 7 · TEACH THE SHELL A WORD THAT SURVIVES A LOGOUT

```
cat ~/.bash_profile ; head -20 ~/.bashrc      # step 5, on Onyx: where does bash read its config?
vi ~/.bashrc                                # step 6: G, o, type the function, Esc, :wq
source ~/.bashrc ; type probe ; probe example.com 80
ssh onyx probe example.com 80                # step 7, from your laptop: no prompt, no terminal
```

QUESTION	ANSWER
Does your <code>~/.bashrc</code> have a <code>[-z "\$PS1"] && return</code> line near the top?	
What did <code>probe example.com 80</code> print?	
After logging out and back in, did probe still work?	
Did <code>ssh onyx probe example.com 80</code> work from your laptop?	

Q2. `ssh onyx probe example.com 80` starts a shell with no prompt and no terminal, and it still found the function. In one sentence: why would that same command **fail** on a machine whose `~/.bashrc` began with the guard line from step 5?

Q3. Step 8: you made **probe2** as an alias and it worked with a URL at the end. Why can an alias not do what **probe host port** does?

Step 9. Same function in your laptop's shell config (`~/.zshrc` on macOS, `~/.bashrc` on Linux and Git Bash), new terminal window, **probe example.com 80**. Which members have it working on the laptop? _____

Graded pass/fail on a row per member and something under each question. Hand this in before you leave.